

Secret Sharing as Side Channel Mitigation

2025

1 Introduction

Secret sharing is a well known technique in cryptography. The basic idea of secret sharing is to split up sensitive variables into several shares. The shares are chosen in such a way that if an adversary gets access to some shares (but not all) they learn nothing about the sensitive value.

We looked at several methods before, and found two that work: Boolean secret sharing (suitable for (n, n) schemes), and Shamir secret sharing (suitable for (t, n) schemes).

This idea also works in the context of leakage: if we secret share intermediate values (including keys), and their shares “look random”, then they may also leak information that “looks random” (i.e. it is unrelated to the sensitive value) and therefore an adversary does not learn any information about the sensitive value. However, the problem in the context with leakage is that a device must have all the shares, and it will compute on all the shares, and therefore it seems clear from the outset that if the adversary attempts to “bring together” leakage information from different shares they will eventually succeed. We analysed before why then secret sharing intermediate values is still interesting: it is because the “bringing shares together” increases the noise variance, and therefore, it changes the SNR very unfavourably for an attacker. In particular, in a scheme with d shares, we increase the noise by an exponential factor in d , which leads to a potentially exponential increase in the number of needed traces for a successful attack. So what is left to argue?

1.1 Security of sharing out additions and multiplication operations

For the sake of simplicity, I will now adopt a terminology that is directly linked with AES implementations. AES represents data via a “state” that consists of 16 state bytes. It represents the key (and round keys) via a “key state” also comprised of bytes.

When we say that we share out intermediate values in an implementation, then what we really mean is that we represent each state byte and each key state byte via multiple shares. This is easy enough to do at the start of the encryption/decryption. But obviously, the state and key state are then worked on: in AES, we apply the round functions multiple times. This is similar to the situation in MPC: we have our inputs (shared out)

and a function (rewritten so that it operates on shares). We needed to think about and argue how to perform an addition and a multiplication on shares. We did this in the context of shares that are elements over $\mathbb{Z}_p$ and that were constructed using Shamir's scheme.

In the context of AES implementations, we will largely work with intermediate values v that represent an element within either state or the key state. In the context of AES it is convenient to understand v as bit string (or element over a finite field with 256 elements, we call this $\mathbb{Z}_{2^n}$ with $n = 8$). That is why Boolean secret sharing is a good choice for securing AES implementations.

2 Boolean (d, d) secret sharing

We now recap on Boolean sharing, generalising it from the $(2, 2)$ setting to working in a (d, d) setting. We share out each (intermediate) value v by generating $d - 1$ random values v_i and exclusive-oring them with v to determine v_d .

```
1 def Bdd.Share(v, d):  
2     for i = 1 upto d - 1  
3          $v_i \xleftarrow{\$} \{0, 1\}^l$   
4      $v_d = v_1 \oplus v_2 \dots v_{d-1} \oplus v$   
5     return  $\{v_1, v_2, \dots, v_d\}$ 
```

Listing 1: Boolean (d, d) secret sharing: Share function

```
1 def Bdd.Reconstruct( $\{v_1, v_2, \dots, v_d\}$ ):  
2     return  $(v_1 \oplus v_2 \oplus \dots \oplus v_d)$ 
```

Listing 2: Boolean (d, d) secret sharing: Reconstruct function

2.1 Security argument

We argued about the security of the $(2, 2)$ Boolean secret sharing scheme before. Now we need to consider the case for (d, d) where $d \geq 2$. Perhaps you can predict that this is not particularly difficult, or different to what we did before.

Our security notion for secret sharing is based on the indistinguishability of shares of two messages m_L and m_R . In the context of an AES implementation, these messages are bytes, but our proof simply asks for bit strings of some fixed length l . We will use

```

1 Share( $m_L, m_R, d, \mathcal{I}$ ):
2   if  $|\mathcal{I}| > d - 1$ :
3     return  $\perp$ 
4   for  $i = 1$  upto  $d - 1$ 
5      $v_i \xleftarrow{\$} \{0, 1\}^l$ 
6    $v_d = v_1 \oplus v_2 \dots v_{d-1} \oplus m_L$ 
7   return  $\{v_i | i \in \mathcal{I}\}$ 

```

Listing 3: Left World

```

1 Share( $m_L, m_R, d, \mathcal{I}$ ):
2   if  $|\mathcal{I}| > d - 1$ :
3     return  $\perp$ 
4   for  $i = 1$  upto  $d - 1$ 
5      $v_i \xleftarrow{\$} \{0, 1\}^l$ 
6    $v_d = v_1 \oplus v_2 \dots v_{d-1} \oplus m_R$ 
7   return  $\{v_i | i \in \mathcal{I}\}$ 

```

Listing 4: Right World

the fact that the one-time pad has indistinguishable encryptions (we showed this before, please refer to the handout on “Adversaries and Proof Techniques”).

Recall that the notion of indistinguishability of shares refers to the following “game”:

- The adversary produces two equal length messages m_L and m_R , and asks for at most $d - 1$ (out of d) shares. If they ask for more, then the game aborts.
- The adversary gets those shares. The adversary wins the game if they can correctly decide (with probability that is larger than randomly guessing) whether the shares come from sharing out m_L or from sharing out m_R .

This game is one way of expressing the property that the shares do not reveal any information about the message that was shared out. Obviously there would be no point in playing this game if the adversary can get all d shares, as this makes reconstruction trivially possible. In the context of implementations, where all shares are manipulated on the same device, this situation is evidently practically possible, if not even likely: this is why we previously looked at the implications on the attack complexity (aka number of needed traces) when we combine trace points (aka shares). Remember that we found that the impact of this is that we increase the adversaries need for traces exponentially by a factor of d . Hence this is a worthwhile strategy from a practical perspective.

Rather than expressing this game textually and with probabilities, we used an algorithmic representation before. We also do this now and produce an algorithm for the “Left” world (which works on message m_L) and an algorithm for the “Right” world (which works on message m_R). Both algorithms are given below. The algorithms take both messages as input, the number of shares d , as well as a set of indices $\mathcal{I}$ indicating which shares they request.

If we can convincingly argue that the Left world and the Right world are indistinguishable from one another, then this proves that an adversary can never win the game. This clearly implies that the two worlds must have the same (number/set of) inputs, and produce the same (number/set of) outputs. This is clearly the case above as the interface of the Left

```

1 Share( $m_L, m_R, d, \mathcal{I}$ ):
2   if  $|\mathcal{I}| > d - 1$ :
3     return  $\perp$ 
4   if  $d \notin \mathcal{I}$ :
5     for  $i = 1$  upto  $d - 1$ 
6        $v_i \xleftarrow{\$} \{0, 1\}^l$ 
7      $v_d = v_1 \oplus v_2 \dots v_{d-1} \oplus m_L$ 
8     return  $\{v_1, \dots, v_{d-1}\}$ 
9   if  $d \in \mathcal{I}$ :
10    for  $i = 1$  upto  $d - 1$ 
11       $v_i \xleftarrow{\$} \{0, 1\}^l$ 
12     $v_d = v_1 \oplus v_2 \dots v_{d-1} \oplus m_L$ 
13    return  $\{v_i | i \in \mathcal{I}\}$ 

```

I have rewritten the Left world so two cases of requested indices are separated out. In the first case, the adversary asks for indices that do not include the final share v_d . The final share v_d is the only share where the computation depends on the input m_L . In the second case, the adversary's request includes the share v_d .

and Right world are exactly the same. Now what is left to argue is that the distribution of shares that the Left world produces is the same as the distribution of shares that the Right world produces. One can do this with arguments from probability theory (we did this before in the context of the the B22.Share function), or, we can try and “rewrite” the Left world into the Right world. We will now follow this latter approach, using the fact that the one-time pad encryption has indistinguishable encryptions.

The first step in doing so consists of separating out two cases that can occur when the adversary chooses the set of indices $\mathcal{I}$.

We can now consider if and why we may be able to change the use of the left input m_L into using the right input m_R . Let us start with the first case, for which I coloured m_L in blue. Notice that in this first case, the share v_d is not returned. Therefore it does not matter whether we have m_L or m_R : and thus we can replace m_L with m_R . Let us now think about the second case where the share v_d is returned to the adversary (typeset in green colour). The question is here if v_d gives any information about m_L : we have a definitive answer to this question, because v_d is a one-time encryption of m_L via the one-time pad. We have proven before that a one-time pad encryption of any m gives no additional information about m . Therefore seeing v_d gives the adversary no information about whether m_L or m_R was used: thus we may also change the green coloured m_L into an m_R and the adversary cannot distinguish the change.

Summarising, we can change both uses of m_L to using m_R , which really is what happens in the Right world. This shows that the Left and the Right world are indistinguishable for an adversary, so we have shown indistinguishability of shares.

2.2 Boolean secret sharing vs AES round functions

We said that we want to apply Boolean secret sharing to AES. This implies that at the start of an encryption¹ we share every single state byte (and every single key byte) into d shares. At the end of an encryption, we perform a reconstruct for all the elements of the state. But what happens in between when we essentially do an “MPC version” of the AES round function? Operations that are linear with regards to the Boolean exclusive-or can be done on shares in a simple manner. Operations that are not linear, are not easy to deal with².

AddRoundKey. It is based on exclusive-oring each state byte with the corresponding key state byte. Let us focus on a single state byte v and a single key byte k then $\text{AddRoundKey}(v, k) = v \oplus k$. When we work with shares, then, conceptually, we are applying AddRoundKey to all pairs of state and key shares. Suppose $v = (v_1, v_2, \dots, v_d)$ and $k = (k_1, k_2, \dots, k_d)$, then $\text{AddRoundKey}(v_i, k_i) = v_i \oplus k_i$ which gives us $\text{AddRoundKey}(v, k) = v \oplus k$ via $\text{BDD.Reconstruct}(\{v_1 \oplus k_1, v_2 \oplus k_2, \dots, v_d \oplus k_d\})$.

ShiftRows. This operation only moves state bytes around, but does not change their value. Therefore no considerations are necessary regarding the impact of Boolean secret sharing.

MixColumns. This is a linear operation with regards to the Boolean exclusive-or because it consists of exclusive-or operations and multiplications with a constant. In particular, considering a $(2, 2)$ scheme where $v_i = (m_i, v_{m_i}) = ((m_i, v_i \oplus m_i))$, then because of the linearity of MixColumns we have that:

$$\text{MC}(v_{m_1}, v_{m_2}, v_{m_3}, v_{m_4}) = \text{MC}(v_1, v_2, v_3, v_4) \oplus \text{MC}(m_1, m_2, m_3, m_4).$$

And therefore

$$\text{BDD.Reconstruct}(\text{MC}(m_1, m_2, m_3, m_4), \text{MC}(v_{m_1}, v_{m_2}, v_{m_3}, v_{m_4})) = \text{MC}(v_1, v_2, v_3, v_4).$$

This fact can be trivially generalised to a (d, d) (with $d > 2$) scheme.

SubBytes. Non-linear operations (with regards to the Boolean exclusive-or) are difficult to deal with. A function f is non-linear with regards to $\oplus$ if $f(x \oplus y) \neq f(x) \oplus f(y)$. The AES SubBytes operation is such a function. It applies a non-linear function S to each byte of the state. Because S is not linear with regards to the exclusive-or, we have

¹I will for simplicity always refer to the process as encryption, but of course, this would equally apply to the process of decryption.

²This is the same situation as in the BGW compiler where we found that addition and multiplication were linear operations with regards to the addition over $\mathbb{Z}_p$, but the multiplication was difficult.

that $S(v \oplus m) \neq S(v) \oplus S(m)$. Now we need to consider ways in which we can still compute $S()$ when having Boolean secret sharing applied to state and key bytes.

If the function S is implemented via a table look-up, then a relatively simple and pain-free work around is possible if $d = 2$. This works as follows. We produce a masked table S_m with the property $S_m(v \oplus m) = S(v) \oplus m'$ (we may choose $m = m'$ although this is often not advisable). Generating such a table is a simple process (we just run through all input values v and store $S(v) \oplus m$ in S_m , and therefore $\text{BDD.Reconstruct}(m, S(v) \oplus m) = S(v)$.

```
1 def compute_Sm(table S, byte m):  
2     for i = 0 to 255 do:  
3          $S_m(i) = S(i \oplus m) \oplus m$   
4     return  $S_m$ 
```

Listing 5: Computing $S_m()$

Whilst this process is simple, we will need to repeat it for all state bytes if they are secret shared with fresh randomness. This implies that we would need to store 16 such tables, which is an absolute no go for embedded systems.

Clearly we need to think about trade-offs between how much fresh randomness we can “afford”, and perhaps consider ways other than a table lookup to implement SubBytes. In the rest of this handout, we will come back to this challenge, and consider such options, starting with the “simplest” and working our way up to “complex” implementation options.

3 AES with efficient $(2, 2)$ secret sharing

Representing each byte by two shares is the perhaps simplest implementation option that can be adopted in practice. Thus each state byte and each key byte is represented by two shares.

Let us consider (again) what security guarantees are given (at best) by a $(2, 2)$ Boolean secret sharing scheme. Per construction, it only gives a security guarantee when no more than a single share “leaks” to the adversary. In relation to differential input attacks this implies, that we get a guarantee that an adversary who considers trace points individually (so does a purely univariate attack, with no pre-processing that combines trace points), can never succeed with recovering any state byte. As a consequence, they also can never recover any key byte. The term “never recover” means that irrespective of how many traces they have, and how they measure, an attack is provably never going to succeed. Obviously, if an adversary does not adhere to the game and gets leakage from both shares, then there is no security guarantee. In fact we know that in such a case, the

adversary can recover the state byte, and with it eventually the corresponding key byte. However, we also know from before, that doing so increases their effort: they will require quadratically more traces (assuming that the SNR was small enough).

Given that the security guarantee is limited to leakage on single shares, there is little point of utilising fresh randomness for the secret sharing of the state and key bytes. Thus we are going to reuse randomness across multiple state bytes and key bytes. This has the added advantage that we do not have double the size of the state (and key state): instead, we are going to “replace” any v with $v \oplus m$ (m being the randomness, often also referred to as the mask), and we store m just once (as all state bytes will be shared out using the same randomness m).

We already discussed what happens when we apply the AES round functions to secret shared state bytes. From this we know, that their actions are either “predictable” or, in the case of SubBytes, we have work around that makes the action predictable. We are thus going to define a “pattern of reuse” (for the randomness) such that we only need to store one extra table to deal with SubBytes, and such that bytes within one column are “refreshed” so that prior to MixColumns they have well-defined randomness which then leads to well-defined randomness after MixColumns. We will next look at how to refresh shares.

3.1 Share refreshing

What is share “refreshing”? The idea is that for various reasons we want to change the randomness in the secret sharing. So, we would like to switch from e.g. a representation using the tuple (m, v_m) (with $v = v_m \oplus m$) to a representation (m', v'_m) (with $v = v'_m \oplus m'$).

```

1  def B22.Refresh((m, v_m), m') :
2      v'_m = v_m ⊕ m'
3      v'_m = v'_m ⊕ m
4      return {m', v'_m}

```

Listing 6: Boolean (2, 2) Refresh function

This refresh process works because $v_m = v \oplus m$, and thus Refresh in line 3 gives $v'_m = (v \oplus m) \oplus m'$ which means that in line 4 we have $v'_m = ((v \oplus m) \oplus m') \oplus m = v \oplus m'$. Therefore the tuple that we return (m', v'_m) is a correct secret sharing of v . Notice that we first apply the fresh randomness m' to the value v_m : this is because we must ensure that our refresh process never reveals any additional information about the value v .

Informal security argument. We want to argue that the Refresh function does not give any additional information about v to an adversary who can request to see a single

intermediate value that occurs in the function (including function inputs and outputs). The intermediate values are $v_m, m, m', v_m \oplus m', (v_m \oplus m') \oplus m$. We are now going to look at each of these values individually, and argue why none of them gives any additional information about v .

Because m and m' were generated independently of v an adversary does not gain any additional information about v if they request either m or m' . What about the other intermediate values? We know, from the property of one-time pad encryption that $v_m = v \oplus m$ gives perfect secrecy, i.e. does not reveal any additional information about v . Next we consider $v_m \oplus m'$: we know that v_m does not give any additional information about v , and $v_m \oplus m'$ inherits this property because m' was sampled independently of v . Equally, $(v_m \oplus m') \oplus m = v \oplus m'$ is a one-time encryption of v (with m' independently sampled from v) and therefore does not reveal any additional information about v .

3.2 Generating randomness for secret sharing the AES state

Each state byte in the i -th row will be shared out using the same randomness m'_i . Because we have four rows, we require four bytes of randomness.

Each byte in the key state will also be represented via two shares. The randomness used for the key bytes in the i -th row is $m'_i \oplus m$. This implies the need for a fifth byte of randomness (the byte m).

Using one further random byte m' , we compute a table $S_m()$ such that $S_m(v \oplus m) = S(v) \oplus m'$. Further more we compute the action of MixColumns on the randomness used in each column via $(m'_1, m'_2, m'_3, m'_4) = \text{MixColumn}(m_1, m_2, m_3, m_4)$.

The six random bytes $(m_1, m_2, m_3, m_4, m, m')$ must be sampled at the beginning of encryption. The table S_m must be computed and the values (m'_1, m'_2, m'_3, m'_4) must be derived.

These choices may seem arbitrary, but we will see in the subsequent sections why these actually make sense.

3.3 Initial secret sharing of the AES state

We need to define a starting point, which we do in the two figures below. This starting point may look somewhat strange, but it will become clear why this is a good way of using the randomness for secret sharing the state bytes.

You might wonder why, in these figures, there is only element shown despite us using a $(2, 2)$ secret sharing scheme. Recall that because we reuse randomness in a well defined way, we do not have to explicitly represent each state byte by two shares. Instead, we know the randomness for each state byte “per construction”. By “we”, I mean the

(a) AES State: Randomness

m'_1	m'_1	m'_1	m'_1
m'_2	m'_2	m'_2	m'_2
m'_3	m'_3	m'_3	m'_3
m'_4	m'_4	m'_4	m'_4

(b) AES Key State: Randomness

$m'_1 \oplus m$	$m'_1 \oplus m$	$m'_1 \oplus m$	$m'_1 \oplus m$
$m'_2 \oplus m$	$m'_2 \oplus m$	$m'_2 \oplus m$	$m'_2 \oplus m$
$m'_3 \oplus m$	$m'_3 \oplus m$	$m'_3 \oplus m$	$m'_3 \oplus m$
$m'_4 \oplus m$	$m'_4 \oplus m$	$m'_4 \oplus m$	$m'_4 \oplus m$

Figure 1: Initial use of randomness

program flow: the randomness is always generated internally on the device and it never leaves the device. For each new encryption, the randomness is newly generated. The figures therefore indicate how the randomness is used. E.g. the top left state byte is indicated to have randomness m'_1 : this means that the secret sharing here is $(m'_1, a_0 \oplus m'_1)$; or the bottom right state byte is indicated to have randomness m'_4 : this means that the secret sharing here is $(m'_4, a_{15} \oplus m'_4)$.

Why does this pattern of using fresh randomness for elements in the same column make sense? Many real world devices show some Hamming distance leakage between subsequent operations on the data bus. In AES it is quite common to store state bytes column wise: i.e. first state byte (a_0 in the notation of the AES standard) is stored in row 1 and column 1, the second state byte is stored in row 2 and column 1, etc. Thus, if there is Hamming distance leakage, then we would have a catastrophic cancellation of randomness if subsequent intermediate values were using the same randomness: $(v_{1,1,m}, m) \oplus (v_{2,1,m}, m) = (v_{1,1} \oplus v_{2,1}, m) \oplus (v_{1,1} \oplus m, m) = (v_{1,1} \oplus v_{2,1}, 0)$.

3.4 AddRoundKey

AddRoundkey exclusive-ors each state byte with its corresponding key byte. Therefore the randomness that we use in the secret sharing is shown in Fig. 2a.

3.5 SubBytes and ShiftRows

Because we are doing our round functions on shares, we must use the table S_m . We defined $S_m(v \oplus m) = S(v) \oplus m'$: the table S_m is specific to representing the input byte as $(v \oplus m, m)$. The SubBytes step consists then of applying S_m to each state byte, which implies that we change our representation to $(v \oplus m', m')$, see Fig. 2b.

(a) After AddRoundKey

m	m	m	m
m	m	m	m
m	m	m	m
m	m	m	m

(b) After SubBytes

m'	m'	m'	m'
m'	m'	m'	m'
m'	m'	m'	m'
m'	m'	m'	m'

Figure 2: Action of AddRound and SubBytes

(a) After Refresh

m_1	m_1	m_1	m_1
m_2	m_2	m_2	m_2
m_3	m_3	m_3	m_3
m_4	m_4	m_4	m_4

(b) After MixColumns

m'_1	m'_1	m'_1	m'_1
m'_2	m'_2	m'_2	m'_2
m'_3	m'_3	m'_3	m'_3
m'_4	m'_4	m'_4	m'_4

Figure 3: Application of MixColumns

3.6 Refresh and MixColumns

Notice that SubBytes changed the randomness from m to m' to avoid issues around Hamming distance leakage but inevitably (we do not want to store more tables than absolutely necessary) we end up in the situation where elements in the same column now have the same randomness. Because of the potential that data that is used in subsequent operations may leak the Hamming distance between the data (which would lead to a catastrophic cancellation of randomness), we must change our randomness using the B22.Refresh function that we defined before.

Recall that MixColumns is a linear operation. Therefore its application to a column with randomness (m_1, m_2, m_3, m_4) leads to the values (m'_1, m'_2, m'_3, m'_4) , which is exactly the randomness that we applied to the state at the beginning of the round! We also ensure that they RoundKey generation preserves the randomness in its function. Therefore at the beginning of the second round, the secret sharing of all state and key bytes is exactly the same as in the first round, etc. Once all rounds have been worked through, we perform B22.Reconstruct on each state byte and retrieve the ciphertext.

3.7 Security argument

First we recall what the adversary is given. Clearly, and as argued before, when recording e.g. power traces the adversary “sees all shares” of all state bytes, key bytes, and steps during encryption. However, there is no point in trying to prove either “indistinguishability of shares” or “no additional information about any v ” when the adversary sees “everything” because we only have a $(2, 2)$ Boolean secret sharing scheme. Per construction, it only gives a security guarantee when an adversary gets a single share.

Thus the only meaningful assumption is that an adversary who gets a single intermediate (aka share), cannot infer the corresponding state byte. The corresponding “game” is depicted in Figure 4. Notice that in the game the secret key is fixed and placed inside the device. So it remains the same over all encryptions. However the randomness m, m', m_i, m'_i is generated inside the device, and therefore it changes with every encryption. The adversary then gets to ask for one single intermediate that is present inside the device. They do this by providing a label j by which an intermediate on the device is identified: $v(j)$ in the picture. The adversary wins the game if they can infer information about v , e.g. by outputting $g(v)$.

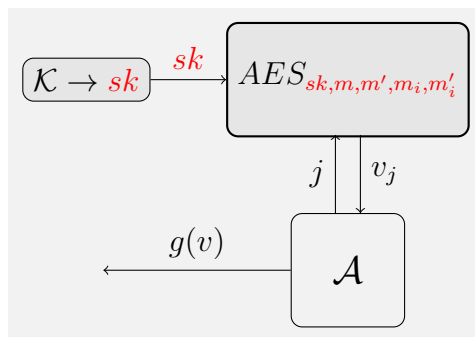


Figure 4: Security game: resilience against adversary who can request a single intermediate value. The adversary wins if they can output depends on v .

We know, based on the properties that we proved before about Boolean $(2, 2)$ secret sharing, that our initial sharing out of state bytes and key state bytes satisfies the game. Therefore we now need to argue that each step in the AES round function also satisfies the game. The intermediate $v(j)$ in the game can be of the form $m, m', m_i, m'_i, v_m, v'_m, v_{m_i}, v_{m'_i}$ and any of the intermediate values that occur during `B22.Refresh()`.

Recall that the random values $m_1, m_2, m_3, m_4, m, m'$ are all independently chosen of all state and key bytes. Therefore, any $v(j)$ that refers to them trivially satisfies the notion of independence. Next we need to reason about the remaining intermediate values.

We already dealt with `B22.Refresh()`. In the context of the round functions `AddRoundKey`, `SubBytes`, `ShiftRows`, we ensure, that each input/output of these is of the form $v \oplus m$

or $v \oplus m'$ or $v \oplus m_i$ or $v \oplus m'_i$. Because all randomness was chosen independently of v , we know from the property of one-time pad encryption that the intermediates are independently distributed of v . The only slightly more interesting function is MixColumns, and intermediate values that have a sharing with randomness m'_i .

In MixColumns the randomness on input bytes m_i are exclusive-ored and multiplied by constants, essentially acting like multiple one-time pad encryptions of the bytes in the column³. Notice though that the values m'_i are not independent of each other any more: this does not matter in because the adversary only gets one intermediate value to work with. Therefore, their dependency cannot be exploited.

Perhaps this security argument is a bit disappointing: but it does resemble what a standard differential input attack is based on. Each trace point is treated in isolation. Furthermore the security argument can also be understood as a “baseline” or “sanity” check for an implementation: any Boolean secret sharing scheme should at least ensure that this standard DPA attack no longer works. And notice that this is an “information theoretic” guarantee (akin to “perfect secrecy”). No matter how many traces an adversary acquires, a standard differential input attack will not work. This is because each intermediate value (seen in isolation) essentially is either a random value or a one-time encryption of something. Therefore the hypotheses that an adversaries can make are independent of how values are represented on the device.

4 AES with (d, d) secret sharing

Using Boolean secret sharing, it is easy to go from a $(2, 2)$ to a (d, d) scheme, when it comes to the **Share** and **Reconstruct** functionality. It should be clear that linear functions, i.e. functions $f()$ with the property that $f(x_1 \oplus x_2 \oplus x_j) = f(x_1) \oplus f(x_2) \oplus f(x_j)$, are again easy to deal with. In the previous section we identified already that AddRoundKey and ShiftRows are just such functions, and that MixColumns is too although it is slightly more computationally demanding. The real problem within AES is the action of the SubBytes function, which is incompatible with Boolean masking. We circumvented this problem by regarding SubBytes as a table-lookup and replacing the troublesome table with another one that “accounts” for the fact that the input to SubBytes is a share of the form $v \oplus m$ (with known and fixed m).

The question is: can we use the same trick when we go from a $(2, 2)$ secret sharing scheme to a general (d, d) secret sharing scheme? The answer will be not efficiently or not at all. Therefore we are going to ask a further question: is there another way of computing SubBytes that is compatible with general (d, d) Boolean secret sharing? And here the answer will be yes.

³I do not write all the intermediate values down here explicitly for the sake of readability.

4.1 SubBytes via table-lookups

Let us look at why SubBytes cannot be done efficiently with table look-ups when using a (d, d) Boolean secret sharing scheme. Consider the case where $d = 3$: i.e. we have $v = (v_1, v_2, v_3 = v \oplus v_1 \oplus v_2)$, and we want to compute a table S' such that $S'_{v_1, v_2}(v_3) = S(v) \oplus v'_1 \oplus v'_2$ (for some independent randomness v'_1, v'_2).

Clearly it is “easy” to compute such a table S' by running through all values of v_1, v_2 and computing $S(v) \oplus v'_1 \oplus v'_2$ and storing them as $S'(v_3)$. This is exactly the same “trick” that we used for $(2, 2)$ Boolean secret sharing.

Here comes the problem: when using a (d, d) Boolean secret sharing scheme, we will obviously want to increase our security guarantee. By this we mean that we would like a provable guarantee that an adversary learns no additional information about v given up to $d-1$ intermediate values. In the $d = 3$ case we would now like to have a guarantee that an adversary cannot succeed when given $d-1 = 2$ intermediate values. This means that no two state bytes ever must use the same S' (aka v_1, v_2). This means that within the same round, we cannot reuse the randomness v_1, v_2 . Therefore, we would need 16 unique tuples (v_1, v_2) and therefore 16 tables S' . Storing 16 tables is not feasible on embedded systems. The next option is then to generate each S' on the fly, which would be way too slow for most applications.

Therefore, the table look-up trick really only works well for $d = 2$ schemes. Extending it to $d = 3$ immediately leads to performance issues. From $d = 4$ onwards, one can even show that producing a table S' cannot be done securely any more⁴.

Another issue that is relevant for the table look-up method is that the computation of S' itself is a potential target for differential input attacks. In the computation of S' we have that v_1, v_2 and v'_1, v'_2 are fixed values (but unknown to the adversary). In the computation of S' we run through all byte values v (most likely in a predictable ordering) in order to compute $S(v) \oplus v'_1 \oplus v'_2$. This means that we have 256 differential inputs (the values of v) and the unknowns v_1, v_2, v'_1, v'_2 . An adversary can attempt to extract information about the v_i , and this has been done in practice⁵.

Summarising, there is little value in trying to push further with the table look-up method. We need another idea/option.

4.2 SubBytes as Function over $\mathbb{Z}_{2^8}$

We have so far ignored what the function $S()$ actually does, by just viewing it as a substitution table, that takes a byte as input and produces another byte as output. But

⁴This is not so straightforward to see, but a nice explanation is given in <https://iacr.org/archive/ches2007/47270028/47270028.pdf> (the open source version of a published paper).

⁵This is part of your summative coursework

how was this substitution table actually generated? The designers of AES argued of course why the choice of $S()$ is a good one, and there are many research papers that discuss this point. A perhaps somewhat accessible representation of what $S()$ actually is, is part of the NIST standard FIPS 197, which defines AES. The standard is available from here this address <https://csrc.nist.gov/pubs/fips/197/final>. It describes how to interpret the state bytes mathematically as elements over a finite field with 256 elements.

SubBytes is defined as the function $S(x) = A \cdot x^{-1} + b$. Here, A is a matrix with elements in $\mathbb{Z}_2$ and b is a vector with elements in $\mathbb{Z}_2$. As a consequence, the non-linearity of S stems from the fact that it is based on a finite field inversion (plus some additions which are linear, so easy to deal with in terms of Boolean secret sharing).

Let us consider the finite field inversion in some more detail. There are two efficient algorithms to compute modular inverses. The first method is based on the so-called extended Euclidean algorithm, and it is not appealing for embedded devices. The second method is based on the fact that $a^{-1} \equiv a^{p-2} \pmod{p}$. This fact is a direct consequence of Fermat's little theorem (check out the background notes if this does not ring a bell). So if $p = 256$, then we can compute a^{-1} by computing a^{254} over $\mathbb{Z}_{2^8}$. This exponentiation can be done via the binary square and multiply method, or, one can think a bit harder and find, what is called an, "optimal addition chain" to represent the exponent 254 and make things a bit more efficient. In any case, the computation of a^{254} will be done via a sequence of finite field square operations $a \cdot a \pmod{p}$ and finite field multiply operations $a \cdot b \pmod{p}$.

Therefore, to implement a $d \geq 3$ Boolean secret sharing for SubBytes, we need to consider how to do a secret sharing version of a multiplication. Because explaining various further implementation tricks/considerations to do a modular multiplication in practice would take too much time, we will now ignore that we are really consider modular multiplications, and focus on just a multiplication instead.

4.3 Multiplications and Boolean secret sharing

We wish to securely compute $c = a \cdot b$ when a and b are secret shared. Thus given $(a_1, a_2, \dots, a_{d-1})$ and $(b_1, b_2, \dots, b_{d-1})$, we wish to compute a secret sharing of $c = a \cdot b$. There are in fact multiple ways to do this, and we will explore at least some of them by firstly looking at the "simplest case" where $d = 2$.

4.3.1 Multiplication for (2,2) Boolean secret sharing

Let us consider what happens if we, straightforwardly, multiply the respective shares with each other (akin to what we did for Shamir secret sharing and multiplications). So we have that $a = (a_1, a_2) = (a_1, a \oplus a_1)$ and $b = (b_1, b_2) = (b_1, b \oplus b_1)$, and we want to compute a $c = (c_1, c_2)$.

Value		
<i>a</i>	a_1	a_2
<i>b</i>	b_1	b_2
$c = a \cdot b$	$a_1 \cdot b_1$	$a_2 \cdot b_2 =$ $a \cdot b \oplus a_1 \cdot b$ $\oplus a \cdot b_1 \oplus a_1 \cdot b_1$

Notice that the sharing of c is given as $c_1 = a_1 \cdot b_1$ and $c_2 = a_2 \cdot b_2$ and sadly, $c \neq c_1 \oplus c_2$. We have the “extra terms” $a_1 \cdot b \oplus a \cdot b_1$. We could actually get rid of these by also computing and adding the cross products $a_1 \cdot b_2 \oplus a_2 \cdot b_1$ (notice that this sum equals exactly the extra terms). Therefore a correct computation would look like described in the table below:

Value		
<i>a</i>	a_1	a_2
<i>b</i>	b_1	b_2
$c = a \cdot b$	$a_1 \cdot b_1$	$a_2 \cdot b_2 \oplus a_1 \cdot b_2 \oplus a_2 \cdot b_1$

But is this secure? Consider the intermediate values that would be generated in an implementation. These would be (assuming we order our computation based on indices, lower come first, remember that most instructions/gates only take two inputs):

$$\begin{aligned}
 v_1 &= a_1 \cdot b_1 \\
 v_2 &= a_1 \cdot b_2 \\
 v_3 &= a_2 \cdot b_1 \\
 v_4 &= a_2 \cdot b_2 \\
 v_5 &= v_4 \oplus v_2 \\
 v_6 &= v_5 \oplus v_3 \\
 \hline
 c_1 &= v_1 \\
 c_2 &= v_6
 \end{aligned}$$

The first four intermediate values are, per construction, independent of a and b , because they are computed using only one share of a and b . The intermediate values $a_2 \cdot b_2 \oplus a_1 \cdot b_2$ and $a_2 \cdot b_2 \oplus a_1 \cdot b_2 \oplus a_2 \cdot b_1$ are both problematic. We can see this in two different ways.

Argument directly via distribution. Let us rewrite the value $a_2 \cdot b_2 \oplus a_1 \cdot b_2$ by grouping terms according to b_2 : $(a_2 \oplus a_1) \cdot b_2 = a \cdot b_2$. At first glance you might argue that we have a kind of “encryption” of a because a is multiplied by b_2 which is uniformly distributed. However, consider what happens if $a = 0$. In that case, no matter what value b_2 takes, the value will always be 0. So clearly the distribution of $a \cdot b_2$ is not independent of the distribution of a . If the adversary request the share $a \cdot b_2$ in the game, they learn from it whether $a = 0$ or not. This violates our security notion. A similar argument can be made for $a_2 \cdot b_2 \oplus a_1 \cdot b_2 \oplus a_2 \cdot b_1$.

Argument via simulation based security. Recall that we used the Real vs Random security notion before to capture the notion of “not learning anything else about something”. The idea here was to construct two worlds; the Real and the Random world. The Real world basically consists of running some algorithm as is. Our probing game is specific to the fact that in the real world, the device holds some secret value(s). Therefore I include the values a, b in the Real world oracle. The Random world is a “simulation” of the Real world. It has the same inputs and produces the same outputs, but it has no access to any of the secrets of the Real world.

In the probing game for $d = 2$, the adversary may request a single intermediate value; for instance $v_1 = a_1 \cdot b_1$. How can the Random world simulate this without access to either a or b ? Quite simply, because a_1 and b_1 are just values drawn randomly from a uniform distribution. Therefore, in the Random world we also just draw two values from a uniform distribution, and return their product to the adversary. Also the intermediate value v_2 can be easily simulated. I give you the Real and Random world as Listings 6 and 7. Notice that in the Real world, there are the values a, b which are not accessible for the adversary. The values a, b are fixed secrets built into the Real world function. Obviously (but not written down explicitly) they are assumed to be bit-strings of length l . The Random world does not have a, b .

But now consider what happens if the adversary requests $a_2 \cdot b_2 \oplus a_1 \cdot b_2$. The value a_1 is again just a random value sampled from a uniform distribution, so this is easy to do. We know that the value b_2 is a one time encryption of b and hence b_2 is indistinguishable from just a random element from a uniform distribution. So we can simulate b_2 easily by drawing an element at random from the uniform distribution. Therefore we can easily simulate $a_1 \cdot b_2$. But we also have $a_2 \cdot b_2$ as part of the expression, which requires us to think about simulating a_2 . We cannot randomly choose a_2 any more because we have already chosen a_1 randomly and a_2 is not independent of a_1 . This means that we really need to have access to a_2 , which of course also implies that we would know a (but in the Random world, we do not). Hence we cannot provide a simulation (the Random world just returns “sorry”), and this means, that the adversary will be able to distinguish between the Real world and the Random world.

Fixing the problem. There is a relatively easy fix for the problem: what we must do is add another “one-time encryption layer” to the computation of v_5 . We do this by adding in a fresh random value z before adding on further terms. Because v_5 is used as part of c_2 the introduction of z only affects c_2 . This would render the secret sharing incorrect, and therefore we must also add z to c_1 . Thus the multiplication would produce the following intermediate values:


```

1 Probe({l}):
2   if |{l}| > 1:
3     return ⊥
4   a1 ←$ {0,1}l
5   b1 ←$ {0,1}l
6   a2 = a1 ⊕ a
7   b2 = b1 ⊕ b
8   v1 = a1 · b1
9   v2 = a1 · b2
10  v3 = a2 · b1
11  v4 = a2 · b2
12  v5 = v4 ⊕ v2
13  v6 = v5 ⊕ v3
14 return vl

```

Listing 7: Real World

```

1 Probe({l}):
2   if |{l}| > 1:
3     return ⊥
4   if l == 1:
5     a1 ←$ {0,1}l
6     b1 ←$ {0,1}l
7     v = a1 · b1
8   elseif l == 2:
9     a1 ←$ {0,1}l
10    b2 ←$ {0,1}l
11    v = a1 · b2
12  elseif l == 3:
13    a2 ←$ {0,1}l
14    b1 ←$ {0,1}l
15    v = a2 · b1
16  elseif l == 4:
17    a2 ←$ {0,1}l
18    b2 ←$ {0,1}l
19    v = a2 · b2
20  else
21    return ‘‘sorry’’
22 return v

```

Listing 8: Random World

$$\begin{array}{l}
v_1 = a_1 \cdot b_1 \\
v_2 = a_1 \cdot b_2 \\
v_3 = a_2 \cdot b_1 \\
v_4 = a_2 \cdot b_2 \\
v_5 = z \oplus v_4 \\
v_6 = v_5 \oplus a_1 \cdot b_2 \\
v_7 = v_6 \oplus a_2 \cdot b_1 \\
\hline
c_1 = v_1 \oplus z \\
c_2 = v_7
\end{array}$$

Now we can easily write a simulation for the Random world, as shown in Listing 9. In Listing 9, we make use of the fact that z is independently drawn (from all other values) from a uniform distribution. Therefore, all the v_i that it is involved in, are in fact one-time pad encryptions of something. Hence it is easy to simulate their distribution, as it is known to be a uniform distribution. Consequently an adversary cannot distinguish the

```

1 Probe({l}):
2   if |{l}| > 1:
3     return  $\perp$ 
4    $a_1 \xleftarrow{\$} \{0,1\}^l$ 
5    $b_1 \xleftarrow{\$} \{0,1\}^l$ 
6    $z \xleftarrow{\$} \{0,1\}^l$ 
7    $a_2 = a_1 \oplus a$ 
8    $b_2 = b_1 \oplus b$ 
9    $v_1 = a_1 \cdot b_1$ 
10   $v_2 = a_1 \cdot b_2$ 
11   $v_3 = a_2 \cdot b_1$ 
12   $v_4 = a_2 \cdot b_2$ 
13   $v_5 = z \oplus v_4$ 
14   $v_6 = v_5 \oplus v_2$ 
15   $v_7 = v_6 \oplus v_3$ 
16 return  $v_l$ 

```

Listing 9: Real World

```

1 Probe({l}):
2   if |{l}| > 1:
3     return  $\perp$ 
4   if  $l == 1$ :
5      $a_1 \xleftarrow{\$} \{0,1\}^l$ 
6      $b_1 \xleftarrow{\$} \{0,1\}^l$ 
7      $v = a_1 \cdot b_1$ 
8   elseif  $l == 2$ :
9      $a_2 \xleftarrow{\$} \{0,1\}^l$ 
10     $b_2 \xleftarrow{\$} \{0,1\}^l$ 
11     $v = a_2 \cdot b_2$ 
12  elseif  $l == 3$ :
13     $a_2 \xleftarrow{\$} \{0,1\}^l$ 
14     $b_1 \xleftarrow{\$} \{0,1\}^l$ 
15     $v = a_2 \cdot b_1$ 
16  elseif  $l == 4$ :
17     $a_2 \xleftarrow{\$} \{0,1\}^l$ 
18     $b_2 \xleftarrow{\$} \{0,1\}^l$ 
19     $v = a_2 \cdot b_2$ 
20  else:
21     $v \xleftarrow{\$} \{0,1\}^l$ 
22 return  $v$ 

```

Listing 10: Random World

Real and Random world: both have the same inputs and outputs, and we have argued why the distributions of v_l (output from the Real world) is the same as the output v from the random world, no matter which intermediate the adversary selects. In cryptography jargon, the game where an adversary can select d intermediate values of an algorithm “for inspection” is also called the **probing security game**. We have now, for the first time, made this game (and the name) explicit. Because we only consider a $d = 2$ Boolean secret sharing, the adversary may only ask for a single intermediate. Notice that in my Real and Random worlds, the adversary can, technically speaking, only probe the v_i ’s. But it is easy to extend my code so that they could of course also ask for a_i , b_i , and c_i values, and they could also ask for z .

4.4 Moving towards $d > 2$

We developed a secure multiplication method for $d = 2$ Boolean masking based on multiplying “corresponding” shares. This was motivated by the fact that we did just this in the context of Shamir secret sharing. But remember: Shamir secret sharing in fact required this strategy because there, the shares are points on a polynomial, and we needed to ensure that we add points with the same x coordinate. Boolean secret sharing is not based on polynomials, and we have the freedom to add terms in “arbitrary ways”⁶.

Example. Another way of performing the shared multiplication from the previous section would be:

$$\begin{array}{l}
 v_1 = a_1 \cdot b_1 \\
 v_2 = a_1 \cdot b_2 \\
 v_3 = a_2 \cdot b_1 \\
 v_4 = a_2 \cdot b_2 \\
 v_5 = z \oplus v_1 \\
 v_6 = v_5 \oplus v_3 \\
 v_7 = z \oplus v_4 \\
 v_8 = v_7 \oplus v_2 \\
 \hline
 c_1 = v_6 = z \oplus a \cdot b_1 \\
 c_2 = v_8 = z \oplus a \cdot b \oplus a \cdot b_1
 \end{array}$$

Clearly, this alternative requires more steps, so it is not a “better” option than what we had before. But it goes to show that in principle, there are multiple ways in which we can introduce fresh randomness, and there are multiple ways in which we can accumulate the intermediate values, and how we assign them to the output shares c_i .

You might wonder why does this matter? Try, as a thought experiment, extending the previous approach to $d > 2$. In the “simplest case”, where $d = 2$, we were able to enumerate all intermediate values that our multiplication algorithm produces, and then, we made reasonable effort to reason about the probing-security game. The effort was reasonable because with $d = 2$, we only allow the adversary to select a single intermediate value. However, if we have $d = 3$, then we would like to strengthen the security guarantee so that an adversary who selects any two intermediate values cannot succeed. Thus, if we keep our proof strategy “as is”, then we would need to reason about all tuples of intermediate values. These would be $\binom{n}{2}$ combinations to consider, whereby n is the number of intermediate values that we have in the multiplication algorithm. So for a multiplication algorithm that takes in d shares for a and b , and that produces n intermediate values, there are $\binom{n}{d}$ d -tuples of intermediate values to reason about! This either asks for a computer aided proof (and yes, there are such schemes out there that come with just

⁶We just saw that the addition of terms can lead to problems, so it is not entirely arbitrary.

such a proof), or, one finds a smart way of writing a multiplication algorithm, so that an elegant mathematical proof is possible. To round off our exploration of real world countermeasures, we will look at such an elegant way.

4.5 ISW multiplication

The abbreviation ISW stands for three cryptographers: Y. Ishai, A. Sahai, and D. Wagner. They came up with a multiplication algorithm for (d, d) Boolean shared inputs (for arbitrary d), that comes with a proof strategy that also works for arbitrary d . By this I mean that their proof strategy does not require the costly enumeration of all combinations of $d - 1$ intermediate values.

I will give the algorithm in its general form first, see Alg. A, and then we look at the “simple cases” for $d = 2$ and $d = 3$, in order to understand how the algorithm, and the associated proof strategy, work.

Alg. A: SecMult with l -bit values

Given input shares $a_1, a_2, \dots, a_d$, with $\bigoplus_i a_i = a$, and $b_1, b_2, \dots, b_d$, with $\bigoplus_i b_i = b$, compute $c_1, c_2, \dots, c_d$, with $\bigoplus_i c_i = c$, and $c = a \cdot b$.

```

1  def SecMult( $\{a_1, a_2, \dots, a_d\}, \{b_1, b_2, \dots, b_d\}$ ):
2      for  $i = 1$  to  $d$  do:
3          for  $j = i + 1$  to  $d$  do:
4               $r_{i,j} \xleftarrow{\$} \{0, 1\}^l$ 
5               $r_{j,i} = (r_{i,j} \oplus a_i b_j) \oplus a_j b_i$ 
6      for  $i = 1$  to  $d$  do:
7           $c_i = a_i b_i$ 
8          for  $j = 1$  to  $d$  do:
9              if  $j \neq i$ :
10                  $c_i = c_i \oplus r_{i,j}$ 

```

I will now “translate” the working principle of the algorithm into something slightly more “graphical”, so that the intuition behind the algorithm becomes clearer. To do so we first revert to the $d = 2$ case, and then we go for the $d = 3$ case.

4.5.1 ISW for $d = 2$

As a brief reminder, when $d = 2$, we have that a_1, b_1 are selected randomly, and $a_2 = a_1 \oplus a$ and $b_2 = b_1 \oplus b$. We have already seen before that the multiplication of $a_2 b_2$ produces “extra terms” that need to be gotten rid off, and we can do this by bringing in the other product terms, i.e. $a_1 b_2$ and $a_2 b_1$. In the table below, I have now written down all the

products that the multiplication algorithm has to produce in order to produce a correct result. The final column in the table indicates how the sum over the partial products in each row will be assigned to the output shares c_i . You should notice, that the table presents a correct, but insecure multiplication.

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_1b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_2b_2$

Remember an observation that we made before: we can assign partial products to any output share. With this observation in mind, I will now “re-assign” partial products to output shares. This re-assignment does not solve the issue of security (yet). Another way of looking at the “re-assignment” is that we are going to “flip” elements in the “upper diagonal” onto elements in the “lower diagonal”.

	b_1	b_2	
a_1	a_1b_1		$c_1 = a_1b_1$
a_2	$a_2b_1 \oplus a_1b_2$	a_2b_2	$c_2 = a_2b_1 \oplus a_1b_2 \oplus a_2b_2$

Remember the trick that we used before to make the sum over partial products secure: we introduced a fresh random value, which we used to “one-time pad encrypt” one of the partial products. The SecMult algorithms uses exactly this trick as well. In the case $d = 2$ it samples exactly one such random value, $r_{1,2}$ in line 3. It actually adds this value to the sum of the products a_1b_2 and a_2b_1 . I am now going to introduce $r_{1,2}$ into the table in the “empty spot” that I created. And like I flipped the element on the upper diagonal before to the lower diagonal, I will also flip the fresh randomness onto the lower diagonal

	b_1	b_2	
a_1	a_1b_1	$r_{1,2}$	$c_1 = b_1a_1 \oplus r_{1,2}$
a_2	$(r_{1,2} \oplus a_1b_2) \oplus a_2b_1$	a_2b_2	$c_2 = a_2b_2 \oplus ((r_{1,2} \oplus a_1b_2) \oplus a_2b_1)$

In Alg. A, the term $(r_{1,2} \oplus a_1b_2) \oplus a_2b_1$ is computed in line 4 and assigned to the variable $r_{2,1}$, indicating its position in the “matrix” of partial products. You can see that in the assignment of partial products to the output shares c_i , this term is indeed there. The process that we went through, to get to this final table above, is more or less how Alg. A works. In the first two for loops, we produce the partial products, and new randomness. We also “flip” the partial products in the upper diagonal onto the lower diagonal by the assignment of the variable $r_{j,i}$ (the flipping of the indices achieves just that). In the final for loop(s), we then assign things to output shares, row by row.

4.5.2 ISW for $d = 3$

Let us repeat this exercise for inputs with three shares. Remember that a_1, a_2, b_1, b_2 are random values and $a_3 = a_1 \oplus a_2 \oplus a$, $b_3 = b_1 \oplus b_2 \oplus b$. Therefore, when we produce the partial product $a_3 b_3$ we will create “extra terms” which we can remove by adding in the partial products $a_i b_j$.

	b_1	b_2	b_3	
a_1	$a_1 b_1$	$r_{1,2}$	$r_{1,3}$	c_1
a_2	$(r_{1,2} \oplus a_1 b_2) \oplus a_2 b_1$	$a_2 b_2$	$r_{2,3}$	c_2
a_3	$(r_{1,3} \oplus a_1 b_3) \oplus a_3 b_1$	$(r_{2,3} \oplus a_2 b_3) \oplus a_3 b_2$	$a_3 b_3$	c_3

Like before we introduce fresh randomness to the partial products and flip the elements in the upper diagonal.

	b_1	b_2	b_3	
a_1	$a_1 b_1$	$r_{1,2}$	$r_{1,3}$	c_1
a_2	$r_{2,1} \oplus a_2 b_1$	$a_2 b_2$	$r_{2,3}$	c_2
a_3	$r_{3,1} \oplus a_3 b_1$	$r_{3,2} \oplus a_3 b_2$	$a_3 b_3$	c_3

You can try and map what you see in this table once more to how Alg. A proceeds. In the first two for loops we essentially create all the fresh randomness (blue values in the table above), and we add the randomness immediately to the partial products that are not on the diagonal. Thereby we do the “flipping” from the upper to the lower diagonal by swapping the indices from $r_{i,j}$ to $r_{j,i}$. Once all these partial products are computed, we produce the output shares by summing up across the rows (and generating the partial products on the diagonal).

4.6 ISW security claim

The idea behind the ISW security proof is based on the simulation based security notion that we used before. A brief recap on this notion: in the Real world, an algorithm is executed with some known inputs and some unknown, secret values. In the Random world, the same algorithm is executed with the same known inputs, but random values instead of the secret values. If there are inputs to the algorithm, then the adversary can provide them. The adversary can choose (with some limitations) some outputs and/or intermediate values. If an adversary cannot tell whether they are interacting with the Real world or whether they are interacting with the Random world, then, as a consequence, there cannot be any information about the secret values in the outputs/intermediate values that the adversary receives in the Real world.

In the context of a multiplication where the inputs a, b are secret shared via a (d, d) Boolean secret sharing scheme, the adversary can choose any $d - 1$ out of d values. We want to show that no matter which $d - 1$ intermediate values the adversary chooses, these intermediate values can always be perfectly simulated by just randomly sampling values (in a specific way) in the Random world, i.e. without ever needing the inputs.

We will now argue that a perfect simulation of $d - 1$ intermediate variables is always possible.

In the ISW multiplication SecMult (see Alg. A), with d input shares of a and independent d input shares of b , any $(d - 1)$ tuple of intermediate variables (including input and output shares), can be perfectly simulated.

The proof statement is the same as saying that any $(d - 1)$ tuple of intermediate variables (including input and output shares) only depends on $d - 1$ shares a and $d - 1$ shares of b . The reason why these two statements are the same is because we have a (d, d) Boolean secret sharing scheme: it has the property that any combination of $d - 1$ shares is independent of the secret value.

4.7 Proof argument

The proof argument is based on the following observations:

- Any intermediate value is produced by either adding or multiplying two values. E.g. $a_1 b_2$, then it is the product of a_1 and b_2 (this now becomes a new intermediate value), so $r_{1,2} \oplus a_1 b_2$ is the sum of the two intermediates $r_{1,2}$ and $a_1 b_2$, etc.
- The intermediate values (and inputs and outputs) that occur above or on the diagonal (in the corresponding matrix representation), i.e. $a_i, b_i, a_i b_i, r_{i,j}$ (with $j \geq i$), use at most the i th share of a and b .
- The intermediate values that occur below the diagonal, i.e. $a_i b_j$ and $r_{i,j} \oplus a_i b_j$ use i -th share of a and the j -th share of b .
- Whilst $r_{i,j}$ and $r_{j,i}$ are, individually taken distributed uniformly, and independently of a_i, a_j, b_i, b_j their joint distribution does depend on these shares.

We will now show that the simulation of any $(d - 1)$ -tuple of intermediate values (these are the terms $a_i, b_i, c_i, a_i b_i, a_i b_j, r_{i,j}, r_{i,j} \oplus a_i b_j$ and sums of $r_{i,j}$ can be done using less than $d - 1$ shares of a and $d - 1$ shares of b . Because these $d - 1$ shares of a and $d - 1$ shares of b are independent of a and b , they can be drawn at random from a uniform distribution.

The adversary selects a $(d - 1)$ tuple of intermediate values. I call this tuple $\mathbf{v}_A$. The first step in the proof argument is that we are going to construct two sets of indices $\mathcal{I}$

and $\mathcal{J}$. The indices in $\mathcal{I}$ list the shares a_i that are in the intermediate values, and the indices in $\mathcal{J}$ list the shares b_j .

In the first step of our proof argument, we are going to make the following assignment for each element v in $\mathbf{v}_A$:

- If v is on or above the diagonal (or a sum of these elements), then we add the index i of the intermediate value to $\mathcal{I}$ and also to $\mathcal{J}$. This makes sense because $a_i b_i$ clearly requires the i -th share from both inputs, and any sum then also requires both shares. For $a_i, b_i, r_{i,j}$ we notice that adding adding i to $\mathcal{J}$ implies that we add a share of b perhaps unnecessarily, but it makes the notation/argument easier and the proof argument will still work.
- If v is below the diagonal, i.e. they select $a_i b_j$ or $r_{j,i} = r_{i,j} \oplus a_i b_j$, then we add i to $\mathcal{I}$ and j to $\mathcal{J}$.

Examples. Suppose that $d = 4$. The adversary can choose a 3-tuple. Here are a few examples.

- $(a_2 b_2, r_{3,2}, r_{2,4})$: then the sets are $\mathcal{I} = \{2, 3\}$ and $\mathcal{J} = \{2\}$
- $(a_1, a_2, r_{4,3})$: then the sets are $\mathcal{I} = \{1, 2, 4\}$ and $\mathcal{J} = \{1, 2, 3\}$
- $(a_1, r_{3,4}, r_{4,3})$: then the sets are $\mathcal{I} = \{1, 3, 4\}$ and $\mathcal{J} = \{1, 3, 4\}$
- $(r_{4,1}, r_{4,2}, r_{2,4})$: then the sets are $\mathcal{I} = \{2, 4\}$ and $\mathcal{J} = \{1, 2, 4\}$

Notice that per construction of the sets $\mathcal{I}, \mathcal{J}$, if both $r_{i,j}$ and $r_{j,i}$ are requested by the adversary, then the indices i, j will occur both in $\mathcal{I}$ and in $\mathcal{J}$.

Importantly notice that both lists can have at most $d - 1$ indices. This is clear because we have at most $d - 1$ intermediate values in our $d - 1$ tuple, and each intermediate value can add at most one index to i and one index to j .

The second step in our proof argument is to show how to simulate the $r_{i,j}$ that occur in the computation of Alg.A. There are three cases that we must consider:

- $r_{i,j} \notin \mathbf{v}_A$: Therefore $r_{i,j}$ is not used and we can simply simulate it by sampling a random value. Notice such an $r_{i,j}$ means that we have $i \in \mathcal{I}$ but $j \notin \mathcal{I}$.
- $r_{i,j} \in \mathbf{v}_A, r_{j,i} \notin \mathbf{v}_A$ (and vice versa): Therefore we can assign it a random value. Notice that in this case we have $(i, j) \in \mathcal{I}$ but $(i, j) \notin \mathcal{J}$.
- $r_{i,j}, r_{j,i} \in \mathbf{v}_A$: Therefore both are used. Notice that now $(i, j) \in \mathcal{I}$ and $\mathcal{J}$. So we proceed as in the algorithm and sample $r_{i,j}$ randomly, and use a_i, b_i, a_j, b_j to compute $r_{j,i}$.

Examples, cont. Consider the same examples as before.

- $(a_2b_2, r_{3,2}, r_{2,4})$: then the sets are $\mathcal{I} = \{2, 3\}$ and $\mathcal{J} = \{2\}$. Both sets have cardinality smaller than d . We assign random values to all $r_{i,j}$ that are not in the list. Considering $r_{3,2}$, because we don't have $r_{2,3}$ included, so we can randomly sample $r_{3,2}$. With the same argument we randomly sample $r_{2,4}$.
- $(a_1, a_2, r_{4,3})$: then the sets are $\mathcal{I} = \{1, 2, 4\}$ and $\mathcal{J} = \{1, 2, 3\}$. We assign random values to all $r_{i,j}$ except for $r_{4,3}$. For this one we notice that its counterpart $r_{3,4}$ is not in the list and so we can in fact also assign a random value to $r_{4,3}$.
- $(a_1, r_{3,4}, r_{4,3})$: then the sets are $\mathcal{I} = \{1, 3, 4\}$ and $\mathcal{J} = \{1, 3, 4\}$. We assign random values to all $r_{i,j}$ except for the two that are requested. Here we are in the (third) case where $r_{3,4}$ and $r_{4,3}$ are requested. We therefore randomly sample $r_{3,4}$ and then use a_3, a_4, b_3, b_4 to assign a value to $r_{4,3}$.
- $(r_{4,1}, r_{4,2}, r_{2,4})$: then the sets are $\mathcal{I} = \{2, 4\}$ and $\mathcal{J} = \{1, 2, 4\}$. We assign random values to all $r_{i,j}$ that are not in the list. We have that $r_{4,1}$ is requested but not $r_{1,4}$ (second case) so we assign a random value to $r_{4,1}$. Then we have $r_{4,2}$ and $r_{2,4}$ (third case), so we randomly sample $r_{2,4}$ and then use a_2, a_4, b_2, b_4 in order to assign a value to $r_{4,2}$.

The third step in our proof argument is to show that we simulate all intermediate values given that we just assigned values to all $r_{i,j}$ (and $r_{j,i}$). This is true because:

- Consider the intermediate values of the form $a_i, b_i, a_ib_i, a_ib_j, r_{i,j}$ (including $j > i$), and sums of them including the c_i as a special case. If the adversary requested them then we have $i \in \mathcal{I}$ and $\mathcal{J}$ and we already assigned values to $r_{i,j}$ in the previous step. So any intermediate value in this list can be perfectly simulated.
- Consider the intermediate values of the form a_ib_j and $r_{i,j} \oplus a_ib_j$. If the adversary requested them, then we have $i \in \mathcal{I}$ and $j \in \mathcal{J}$. So we can simulate a_ib_j and we did already assign values to $r_{i,j}$ in the previous step. Therefore we can perfectly simulate these intermediate values as well.

Consequently, we have shown how to assign values to all intermediate variables, and therefore also the ones that the adversary requested. Because $\mathcal{I}$ and $\mathcal{J}$ can contain at most $d - 1$ indices, we can in fact assign random values to all a_i and b_j that are required in the assignments when writing a simulator for the Random world. The distribution of the adversarially chosen tuple $\mathbf{v}_A$ as produced by the Real world will therefore be exactly identical to the distribution of that tuple as produced by the Random world. This concludes our argument.

4.8 Utilising the ISW multiplication

We arrived at the need for a multiplication algorithm that works on (d, d) shared inputs because we want to securely compute the AES S-box. The S-box can be understood as a function that computes a multiplicative inverse; this inverse can be done via a sequence of square and multiply operations. So within the context of AES then, we would do all operations except for the S-Box, by just additively manipulating the d shares of the respective intermediate values. When it comes to the S-Box (and yes, we will need to do 16 of them), we have to ensure that the two inputs to the ISW multiplier are based on independent secret sharings (no re-using of randomness). So refreshing of randomness is required. There are plenty of pitfalls when translating this into a practical implementation, and thus, cryptographic engineers who work in this space do need to understand the details!